Fake vs Real News Classification using Natural Language Processing

INTRODUCTION

Distinguishing between real and fake news has become increasingly difficult as misinformation spreads rapidly across digital platforms. Manual checking is no longer practical as it is too slow, subjective, and unable to keep pace with the amount of information produced each day.

There is now a clear need for automated AI systems that can analyze text and detect patterns of misleading or unreliable articles using natural processing language and machine learning. Developing such models, however, requires careful attention to several challenges, such as data quality, potential bias, avoiding overfitting and underfitting, and ensuring fairness in predictions. Some approaches, especially deep learning approaches, are not also easily interpretable, and may require substantial computational resources. Ethical concerns are also important, as misclassifications, especially when fake news is predicted as real, can lead to the spread of harmful or misleading information. Responsible use and the potential impact of automated decisions further highlight the need for thoughtful design when building AI systems for fake news-detection.

LITERATURE REVIEW

- ***Combating Fake News in the Digital Age: A Review of AI-Based Approaches*** *by Vishnupriya et. al. Published in 2024 IEEE 9th I2CT:*[i]*[1]*
  This conference paper examines the different strategies of detecting false news, highlighting their strengths and weaknesses. It compares the effectiveness of traditional methods with recent techniques that involve AI and machine learning, highlighting the importance of combining both approaches. It discusses some of the challenges involved with using AI techniques, such as data bias and evolving attacks. Furthermore, some emerging strategies are listed that can combat the challenges. In the conclusion, the paper emphasises the importance of continuous development of technologies while simultaneously taking ethical considerations into account.

- ***A Fake News Detection System based on Combination of Word Embedded Techniques and Hybrid Deep Learning Model*** *by OUASSIL et. Al. Published in IJACSA, Vol. 13 Issue 10, 2022:*[ii]*[2]*
  This paper proposes a method of fake news detection combination of different word embedding techniques and a hybrid Convolutional Neural Network (CNN) and Bidirectional Long Short-Term Memory (BILSTM) model. The framework involves four steps: first, preprocessing of the input data; second, converting into word embedding vectors based on the concatenation of two word-embedding techniques and representing them with four different methods; third, training the hybrid model; and fourth, evaluating and comparing the different application methods. This method was implemented using the WELFake dataset.

- ***AI-Based Fake News Detection Using NLP*** *by Omkar Reddy Polu Published in IJAIML Volume 3, Issue 2, 2024:* [iii]*[3]*
  This paper introduces a model for fake news detection that combines AI and NLP methods with deep learning, graph-based learning and explainable AI techniques with the goal of improves accuracy of detection. It highlights how each technique helps the robustness of the algorithm. The proposed methodology is executed in several stages: collecting the data, data cleansing, feature extraction, model training and evaluation. They used the LIAR or FakeNewsNet or ISOT datasets which contain labelled real and fake news. The outcome produced high accuracy; however, the study acknowledges that there are still limitations to the approach.

- ***Can AI Outsmart Fake News? Detecting Misinformation with AI Models in Real-Time*** *by Gregory Gondwe Published in Sage Journals, 2025:* [iv]*[4]*
  The research examines the applications of artificial intelligence to identify misinformation and fake news in real-time. Based on a hybrid approach that integrates machine learning (ML), natural language processing (NLP), and deep learning (DL), the authors produced and validated models based on a dataset of 10,000 classified items (true statements, false statements, uncertain statements). The results demonstrate that transformer models compared to conventional machine learning algorithms, like SVM, Naïve Bayes, and Random Forest, attained substantially improved accuracies in terms of values for accuracy, precision, and recall. The study also flags practical challenges like deploying highly complex models in real-time settings which creates resource concerns. Quality of data (fact-checked data vs unverified data), in terms of training data's contribution to model accuracy, are other areas. The serious ethical concerns related to bias within machine learning models motivated the authors to call for ongoing model optimization, continuous retraining, and increased focus on XAI to support transparency.

- ***A Survey on Natural Language Processing for Fake News Detection*** *by Oshikawa et al. Published in LREC, 2020:* [v]*[5]*
  The paper discusses the use of NLP methods and AI algorithms to detect fake news based on text analysis. The authors describe how fake news detection can be considered a text classification task, where machine learning algorithms are trained to distinguish real and fake content. The process begins with data preprocessing including tokenization, lemmatization, feature extraction using TF-IDF or Word Embeddings techniques. The study compares traditional machine learning algorithms like SVM, Naïve Bayes, Logistic Regression with deep learning algorithms like CNNs or LSTMs. They note how neural networks generally perform better since they are capable of automatically learning complex linguistic features. They go on to describe how attention mechanisms, memory networks, and rhetorical structures can improve accuracy. Several benchmark datasets, such as LIAR, FEVER, and FakeNewsNet are examined, and results show that hybrid models combining text with metadata perform best. They conclude that though AI systems based on NLP are efficient in fake news detection tasks, their performance still depends heavily on dataset quality, model design, and language diversity.

DATA PREPARATION

Before any AI algorithms can be used on the data, the dataset must be prepared so that we can remove unnecessary variables and format it to be understood by the AI. We used the dataset ISOT Fake News Dataset provided by the instructor and can also be found on kaggle[vi][6].

The first step involves loading all the required libraries for text processing. This is just so that NLTK can download tokenizer models, stopword lists, and the WordNet database, which are used for tokenization, stopword removal, and lemmatization.

```
#Import necessary libraries
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tab')
```

We loaded 200 fake and 200 true news articles using pandas. A label is added to each dataset: 0 for the fake news and 1 for the real news. Both datasets are merged into one table with only the important columns.

```
#read datasets, ensure datasets available
fake_df = pd.read_csv("Fake.csv", nrows=200, quotechar='"',
doublequote=True, engine='c')
true_df = pd.read_csv("True.csv", nrows=200, quotechar='"',
doublequote=True, engine='c')

# Create labels for each news: 0 = Fake, 1 = Real
fake_df['label'] = 0
true_df['label'] = 1

# Combine both files
df = pd.concat([fake_df, true_df], axis=0).reset_index(drop=True)
df = df[['title', 'text', 'label']] #give columns names
df.head() #display first 5 rows
```

Data before preprocessing:

	title	text	label
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...	0
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...	0
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...	0
3	Trump Is So Obsessed He Even Has Obama's Name...	On Christmas day, Donald Trump announced that ...	0
4	Pope Francis Just Called Out Donald Trump Dur...	Pope Francis used his annual Christmas Day mes...	0

Next, each news article is cleaned to prepare the data for modeling. The function lower() first converts the text to lowercase, and re.sub() removes any punctuation, numbers and special characters using a regular expression. The function also splits the text into individual words, removes common stopwords, and applies lemmatization, reducing words to their base form. Once the function is written, it is applied to every article in the dataset, creating a new column (clean_text). After the function is defined, the original text is displayed next to its cleaned version that appears lowercase, simplified and free of unnecessary words.

```
import re #support for regular expressions

stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    # Lowercase
    text = text.lower()
    # Remove punctuation, numbers, and special characters
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize
    tokens = nltk.word_tokenize(text)
    # Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]
    # Lemmatization
    tokens_lemmatized = [lemmatizer.lemmatize(word) for word in tokens]
    return " ".join(tokens_lemmatized)

# Apply preprocessing to the news text
df['clean_text'] = df['text'].apply(preprocess_text)
df[['text', 'clean_text']].head()
```

Data after:

	text	clean_text
0	Donald Trump just couldn t wish all Americans ...	donald trump wish american happy new year leav...
1	House Intelligence Committee Chairman Devin Nu...	house intelligence committee chairman devin nu...
2	On Friday, it was revealed that former Milwauk...	friday revealed former milwaukee sheriff david...
3	On Christmas day, Donald Trump announced that ...	christmas day donald trump announced would bac...
4	Pope Francis used his annual Christmas Day mes...	pope francis used annual christmas day message...

CountVectorizer() converts the cleaned text into a matrix of token counts, also referred to as a Bag-Of-Words. This conversion into a numerical format is one the algorithm can understand and process. The top 5000 words are limited to keep the model efficient.

```
bow_vectorizer = CountVectorizer(max_features=5000)
X_bow = bow_vectorizer.fit_transform(df['clean_text'])

print("BoW shape:", X_bow.shape)
```

Next, TfidfVectorizer(max_features=5000) is initialised to calculate TF-IDF (Term Frequency-Inverse Document Frequency) scores for the top 5,000 terms, and then fit_transform() takes all the cleaned texts and converts into a sparse matrix. Each value signifies the importance of the word in a document; higher weight shows that the word is rare or important and lower weight shows that it is common.

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf_vectorizer.fit_transform(df['clean_text'])

print("TF-IDF shape:", X_tfidf.shape)
```

Output:

Training samples: (320, 5000)

Test samples: (80, 5000)

```
#create separate training and testing datasets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, df['label'], test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape)
print("Test samples:", X_test.shape)
```

```
print("Shape of X_train:", X_train.shape)
print("Type of X_train:", type(X_train))
```

Output:

Shape of X_train: (320, 5000)

Type of X_train: <class 'scipy.sparse._csr.csr_matrix'>

MACHINE LEARNING MODEL 1: NAIVE BAYES

The first traditional ML model we implemented in our model was the Multinomial Naïve Bayes classifier. It is based on Bayes' Theorem and works by learning how frequently words appear in each class.

In the context of our project, the model was implemented using the MultinomialNB class from the sklearn.naive_bayes library. The model was trained using fit(), where it learned the probability distributions of words for Fake and Real news. During prediction, the classifier uses these probabilities and multiplies them to determine which class (0 or 1) is most likely for each new article. After training, we ran the model on the test data to see how it classified the unseen articles.

```
# Train the model
nb = MultinomialNB()
nb.fit(X_train, y_train)

# Test the model
y_pred = nb.predict(X_test)
```

MACHINE LEARNING MODEL 2: RANDOM FORESTS

The other traditional ml model we selected was the random forest classifier model. In this model, a group of decision trees are created during the training process, each of which was trained on a random subset of the data. The result of these trees is averaged to improve the accuracy.

The model was implemented using RandomForestClassifier from the sklearn.ensemble library. The number of trees was set to 500 and random_state, the parameter that determines the randomness of the training process, to 50. The split data was fit into the model to train it, and then the model was tested using predict().

```
#train model
model = RandomForestClassifier(n_estimators=500, random_state=50)
model.fit(X_train1, y_train1)

#test model
y_pred = model.predict(X_test1)
```

DEEP LEARNING MODEL: CNN

Our chosen NLP deep learning model is a basic Convolution Neural Network (CNN). It works by sliding a small filter matrix called the Convolution layer across the numerical data matrix, performing a dot product to extract patterns in the data. The most important features are pooled together and passed to the fully connected layer that performs the final predictions.

```
# Convert sparse matrices to dense arrays for Keras
cv_train = X_train1.toarray()
cv_test = X_test1.toarray()

#Define model
model = Sequential()
model.add(Dense(units = 100 , activation = 'relu' , input_dim = cv_train.shape[1]))
model.add(Dense(units = 50 , activation = 'relu'))
model.add(Dense(units = 25 , activation = 'relu'))
model.add(Dense(units = 10 , activation = 'relu'))
model.add(Dense(units = 1 , activation = 'sigmoid'))

#compile model
model.compile(optimizer = 'adam' , loss = 'binary_crossentropy' , metrics = ['accuracy'])

#fit model
print("\nTraining CNN model...")
history = model.fit(cv_train, y_train1, epochs = 10, verbose=0) # Set verbose to 0 to suppress output per epoch
```

First, the training and testing sets are converted into matrices as CNNs work with numerical matrices. Then, the model is defined using Sequential() from tensorflow.keras.models, which forms a linear stack of layers to form a Keras model. Then, five dense (fully connected) layers are added, each with a decreasing number of neurons, from 100 to 50 to 25 to 10 to 1. The hidden layers use a Rectified linear unit activation function, and the final output layer uses the sigmoid activation function, providing a binary output. The argument input-dim informs the number of features in our TF_IDF vector.

Next, the model was compiled, specifying an ‘adam’ optimizer (which adaptively adjusts the learning rate for each parameter based on its historical gradients), a loss function ‘binary_crossentropy’ (which measures the difference between the predicted probabilities and the actual true labels and a metric result ‘accuracy’ (which outputs the model accuracy).

Finally, the model was trained using the training matrix. The parameter epochs specifies the number of passes through the entire training dataset while verbose is set to zero so no output is generated during training.

EVALUATION

After testing each model, we implemented a code that would evaluation the model while also providing visualisation.

NAIVES BAYES

The Naive Bayes model produced the same accuracy and evaluation metrics across three runs, achieving 95% accuracy, which shows that the model is stable and reliable.
The repeated results suggest that the training and testing process is dependable and that the model's performance does not fluctuate under repeated evaluation.

Run	Accuracy
1	0.95
2	0.95
3	0.95

Average = 0.95

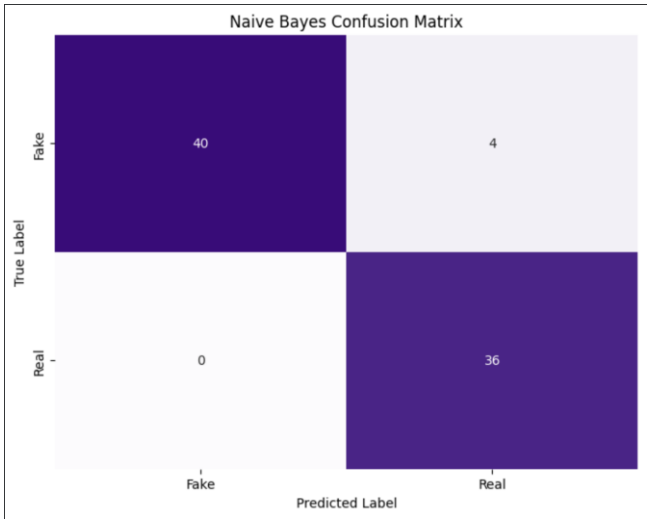
The Multinomial Naive Bayes classifier performed strongly on the fake-real news classification task with an overall accuracy of 95% on the test set. According to the classification report, the Fake class achieved a precision of 1.00, recall of 0.91, and F1 score of 0.95. The Real articles achieved precision of 0.90, recall of 1.00, and F1 score of 0.95. This shows the model was particularly reliable when predicting Fake articles, as it never incorrectly labeled a Real article as fake. Furthermore, it also had a strong sensitivity to Real articles and identified all of them as real. Overall, the model showed balanced and consistent performance across both classes.

```
Accuracy: 0.95

Classification Report:
              precision    recall  f1-score   support

        Fake       1.00      0.91      0.95        44
        Real       0.90      1.00      0.95        36

    accuracy                           0.95        80
   macro avg       0.95      0.95      0.95        80
weighted avg       0.96      0.95      0.95        80
```



The confusion matrix demonstrates that the model classified 40 Fake articles and 36 Real articles correctly. The only errors occurred when the model misclassified 4 Fake articles as Real, resulting in a small number of false positives for the Real class. More importantly, there were no false negatives for Real news, meaning that the model successfully identified every Real article. This confirms the relatively high recall for the Real class and reflects the model's strong ability to avoid mistakenly labeling genuine news as fake.

RANDOM FORESTS

The accuracy of the random forest model varied depending on the training and testing data size split and the random states set in when splitting the data. Most of the time, the accuracy was above 90%, but certain values brought the accuracy much lower. The following are some of the runs performed to illustrate this variation:

test_size (of total data)	random_state (data)	n_estimators	random_state (model)	Accuracy
0.2	20/50/100	500	50	1.0
0.4	20/50	500	50	1.0
0.4	100	500	50	0.994
0.5	50	500	20/50	1.0
0.5	20/100	500	50	0.995
0.7	100	100	50	0.996
0.7	100	100	100	0.992
0.7	100	100	10	0.986
0.9	50	500	50	0.992
0.9	20	500	50	0.511

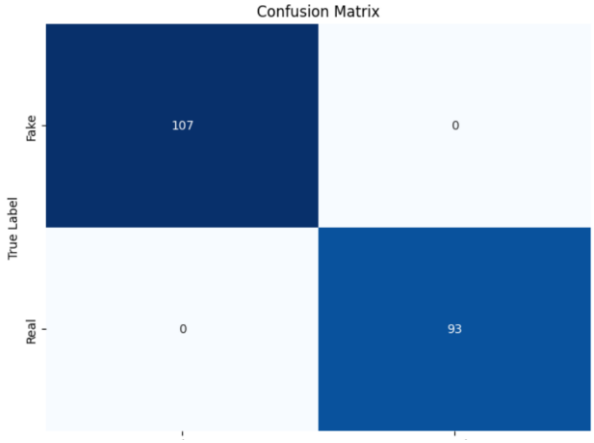
From this table, we can see that lower testing data sizes tended to provide greater accuracy. This is likely the result of the model having a greater training data size, allowing it to better recognise the patterns that differentiated fake news from real news. Changing the random states when splitting the data also had an effect. This is most noticeable in the last two rows, as there is a significant difference in accuracy despite all else being equal. Altering the size of n_estimators and random_state in the model did not cause much deviation in accuracy.

By testing different values, we finalised on 0.5 for test_size, 50 for random_state while splitting, 500 for n_estimators and 50 for random_state in the model. It produces a consistent 100% accuracy across multiple runs while also maximising the testing sample size to properly test the model.

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       107
           1       1.00      1.00      1.00        93

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```



CNN

For the CNN model, we also experimented with different testing sizes and random_states when splitting the data. Moreover, we tried different epochs to explore which values resulted in greater accuracy.

test_size (of total data)	random_state	epochs	Mean accuracy (of 4)
0.2	50	1	0.716
0.2	50	2	0.738
0.2	50	4	0.922
0.2	50	5	0.922
0.2	50	6	0.931
0.2	50	8	0.934
0.2	50	10	0.9375
0.5	50	6	0.845
0.8	50	6	0.827
0.2	20	6	0.9625
0.2	100	6	0.981

When changing the epochs, the accuracy of each run of the code resulted in wildly different values. For example, when epoch is 1, the accuracy could be as high as 0.9 or as low as 0.3. On the other hand, when epoch is 10, the accuracy is a consistent 0.9375 across all runs. The lower the values, the greater the variance in accuracy across multiple runs. Hence, for a consistent accuracy while still being cautious of overtraining, we select the epoch to be 6 for the final evaluation.

Next, we tested how changing the training size to testing size ratio affected the results. When the testing size of the data was increased and the training size decreased, the accuracy decreased. Therefore, we kept the testing size as 0.2 for the highest accuracy.

Then, we changed the random_state when splitting the data. Decreasing the value from 50 to 20 not only increased the accuracy, but it also provided a consistent accuracy score across all four runs. In contrast, while increasing the random_state to 100, the mean accuracy is greater, but the results show a minor variation across runs. For the greatest mean accuracy across all, we selected on the value of random_state as 100.

The results are as follows (when epoch = 6, test_size = 0.2, random_state = 100):

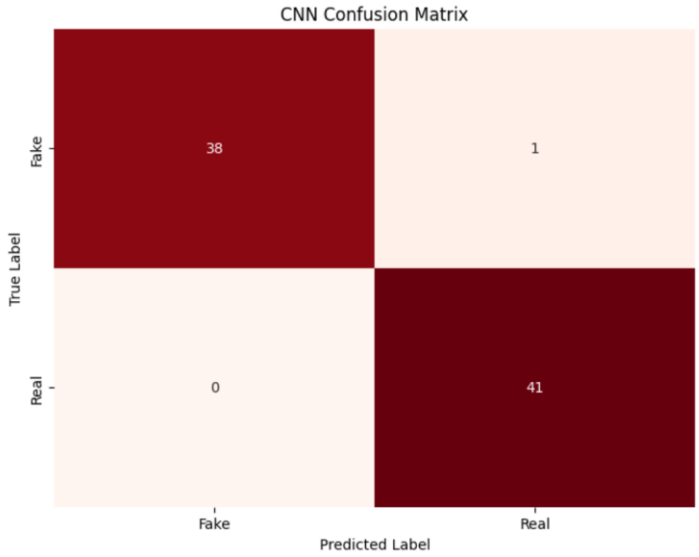
Mean accuracy = 0.981

Mean precision = (0)0.985, (1)0.965

Mean recall = (0)0.96, (1)0.99

Mean f1 score = (0) 0.98, (1)0.985

Example confusion matrix:



CONCLUSION

In this project, we developed and tested several AI models to classify real and fake news articles using Natural Language Processing Techniques. The results showed that traditional models like Naive Bayes and Random Forest performed strongly; Indeed, Random Forest was found to have performed better than the other models with the highest overall accuracy among them all. The CNN model also showed competitive results after tuning factors like epoch size, test size, and random state. It produced results that were comparable to the other models. Therefore, both classical machine learning models and deep learning methods can be effective in detecting fake news articles when properly trained and optimised.

The report also highlights the importance of data quality, fairness, and responsible use, as misclassification can contribute to the spread of misinformation. This shows that while AI can analyse and filter large volumes of news; it also highlights how critical it is to carefully design AI models and to use ethical considerations during its implementation.

1 [i]https://ieeexplore.ieee.org/abstract/document/10544008

2[ii] https://thesai.org/Publications/ViewPaper?Volume=13&Issue=10&Code=IJACSA&SerialNo=61

[3][iii] https://d1wqtxts1xzle7.cloudfront.net/121676027/IJAIML_03_02_019-libre.pdf?1741223500=&response-content-disposition=inline%3B+filename%3DAI_BASED_FAKE_NEWS_DETECTION_USING_NLP.pdf&Expires=1762702726&Signature=Oz0~v5VB2tQC-KzyORFFNhOk-6dZf3j3OaN2w1BdjowhHzTb8dFc2TD46vypkYfeLSGkqODnQgTwAbBOdTQhLfs8DPyBJA5MIYh8AwAxy1~btwDq9VM~26xJrgx8ES5QHCSacax9-borlODWQ83ArgOnJnlccgqn3Z7orZ~nRr~PeHIEEeUe2dxjMNh9FrtT9fRfwld9DbUKKeTtKCkPTY50QGAOglSoEZvEMDnm9~O2ibW-GbeKGNdhP6VOa-i9cTz9jmPBZy~XKX-oqfwnu-dMJZNNj8gnHZ1dE-vMj0g37~dYFY~CnIEJZ3~FDo6-zc4TEvqbCud3eA8bP0iqeQ__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA

- [4] [iv] ***https://journals.sagepub.com/doi/full/10.1177/27523543251325902***

[5] [v] https://aclanthology.org/2020.lrec-1.747.pdf
[6] [vi] https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets/data